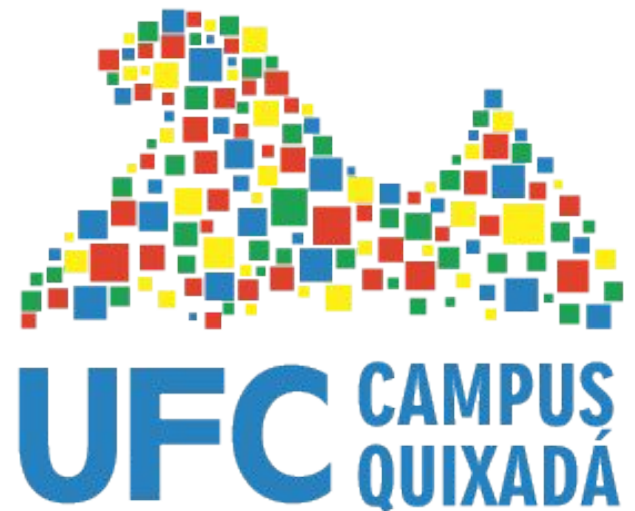


Clean Architecture

Sidarta Carvalho



Robert C. Martin Series

Clean Architecture

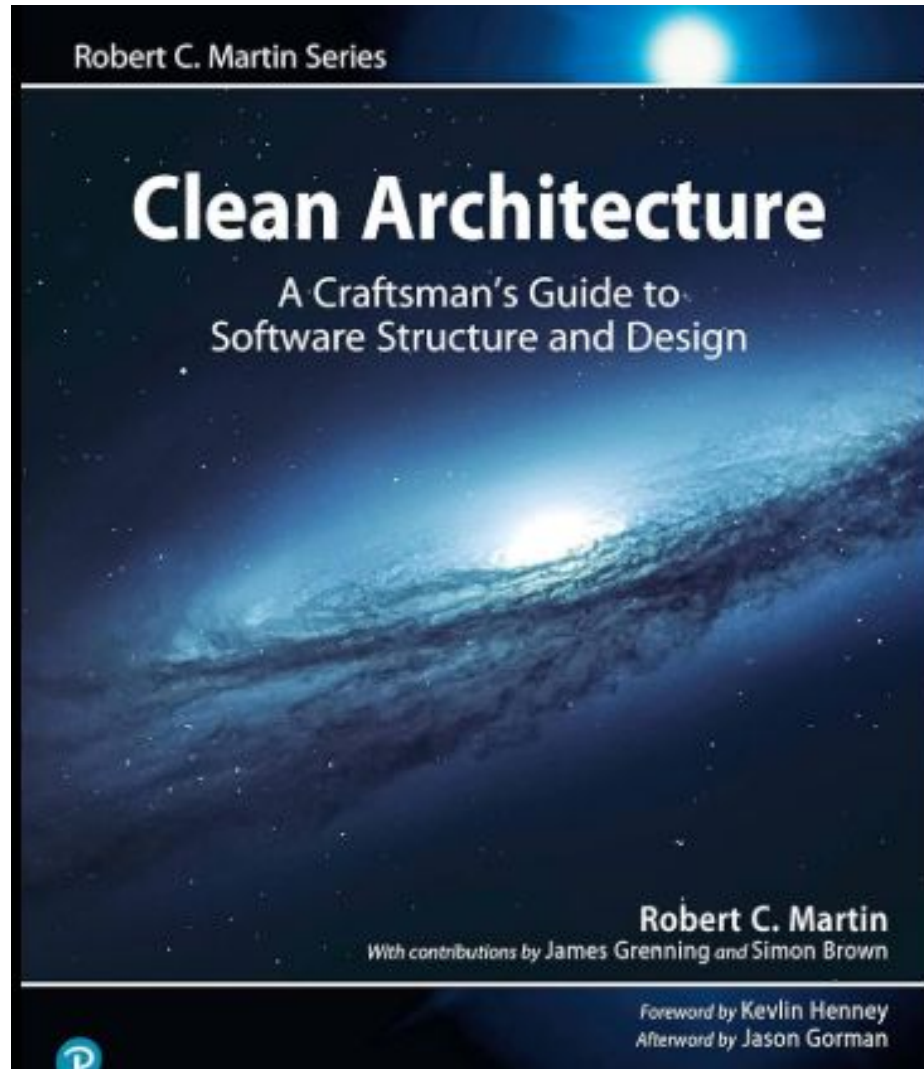
A Craftsman's Guide to
Software Structure and Design

Robert C. Martin

With contributions by James Grenning and Simon Brown

Foreword by Kevlin Henney

Afterword by Jason Gorman



Por que Precisamos da Arquitetura Limpa?

1. Imagine que você entra em um novo projeto.
2. A equipe é grande e está crescendo, o que parece um sinal de sucesso.
3. No entanto, você rapidamente percebe uma verdade frustrante
 - Apesar de todos trabalharem longas horas, a produtividade está despencando.
 - Fazer uma pequena alteração leva semanas, quebra outras partes do sistema e gera um estresse enorme.

Isso não é um pesadelo hipotético. É a realidade da maioria dos projetos de software e é a principal motivação para a Clean Architecture.

O problema não são os desenvolvedores, é a estrutura (arquitetura) do software.

O Custo da Desordem: Um Caso Real

1. Sucesso Aparente

A equipe de engenharia cresce exponencialmente a cada novo lançamento do produto. A gestão está investindo pesado.

2. A Realidade Sombria

Ao mesmo tempo, a quantidade de código novo adicionado a cada lançamento diminui drasticamente, quase parando.

3. O Desastre Financeiro

O custo por linha de código dispara.

No início, a empresa gastava pouco para gerar muita funcionalidade.

No final, gastava 20 milhões de dólares por mês para não gerar quase nada de novo.

Clean Architecture

O Custo da Desordem: Um Caso Real

O que aconteceu?

A equipe caiu na maior mentira do desenvolvimento de software: **"Podemos limpar tudo depois, primeiro temos que entregar."**

A verdade é que o "depois" nunca chega, porque a pressão do mercado nunca diminui.

A bagunça se acumula até que todo o esforço da equipe é consumido apenas para gerenciá-la, em vez de criar valor.

Clean Architecture: Exemplo 1

Uma boa arquitetura protege o coração do seu sistema (as regras de negócio) das coisas que mudam constantemente (a tecnologia).

Exemplo: A Mudança da Interface do Usuário (UI)

Cenário: Você tem uma aplicação desktop de sucesso.

O marketing decide que agora é preciso ter uma versão web.

Com uma Arquitetura Ruim

- Sua lógica de negócio está misturada com o código da UI desktop (janelas, botões, etc.).
- Para criar uma versão web, você precisa reescrever boa parte do sistema, com um risco enorme de introduzir bugs na lógica que já funcionava.
- A UI não é um detalhe, é um emaranhado no núcleo do seu código.

Clean Architecture: Exemplo 1

Uma boa arquitetura protege o coração do seu sistema (as regras de negócio) das coisas que mudam constantemente (a tecnologia).

Exemplo: A Mudança da Interface do Usuário (UI)

Com a Clean Architecture

- Suas regras de negócio (as "Entidades" e "Casos de Uso") não sabem o que é uma UI.
- As regras de negócio são o centro do sistema.
- A UI desktop é apenas um "plugin" que chama essas regras.
- Para criar a versão web, você simplesmente cria um novo plugin (um conjunto de controllers e views web) que chama as mesmas regras de negócio, que permanecem intocadas e estáveis.
- A web é tratada como o que ela é: um detalhe de entrega.

Clean Architecture: Exemplo 2

Exemplo: A Troca do Banco de Dados

Cenário: Sua aplicação usa um banco de dados relacional (ex: MySQL).

Anos depois, por questões de escala e desempenho, a empresa decide migrar para um banco de dados NoSQL.

Com uma Arquitetura Ruim

- Suas regras de negócio estão cheias de comandos SQL.
- Seus objetos de negócio dependem diretamente de um framework ORM (Object-Relational Mapper).
- Trocar o banco de dados se torna uma tarefa monumental que exige reescrever o sistema quase do zero, impedindo o desenvolvimento de novas funcionalidades por meses.

Clean Architecture: Exemplo 2

Exemplo: A Troca do Banco de Dados

Cenário: Sua aplicação usa um banco de dados relacional (ex: MySQL).

Anos depois, por questões de escala e desempenho, a empresa decide migrar para um banco de dados NoSQL.

Com a Clean Architecture

- Suas regras de negócio não sabem qual banco de dados está sendo usado, nem mesmo se ele é relacional.
- Elas se comunicam com uma abstração, uma interface (por exemplo, `IRepository`).
- O banco de dados é um "plugin" que implementa essa interface.
- Para migrar, você escreve um novo plugin para o NoSQL que implementa a mesma interface.
- A mudança ocorre na camada mais externa da arquitetura, e o núcleo do seu sistema (onde o valor real reside) não é afetado.

Clean Architecture: Exemplo 2

Exemplo: A Troca do Banco de Dados

Em resumo, a motivação para a Clean Architecture é a sobrevivência e a sustentabilidade de um projeto.

Ela é uma estratégia para garantir que o software permaneça "soft" — ou seja, fácil de mudar — protegendo a lógica de negócio das decisões tecnológicas que, inevitavelmente, se tornarão obsoletas.



Clean Architecture

Clean Architecture é um conjunto de princípios de design de software cujo objetivo principal é a separação de preocupações.

Ela faz isso dividindo o software em camadas, permitindo a criação de sistemas que são

Independentes de frameworks

A arquitetura não depende de bibliotecas de software, permitindo que você as use como ferramentas, em vez de forçar seu sistema a se adaptar a elas.

Testáveis

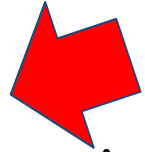
As regras de negócio podem ser testadas sem a necessidade da interface do usuário (UI), banco de dados, servidor web ou qualquer outro elemento externo.

Independentes da interface do usuário

A interface do usuário pode ser facilmente alterada sem impactar o resto do sistema.

Independentes de banco de dados

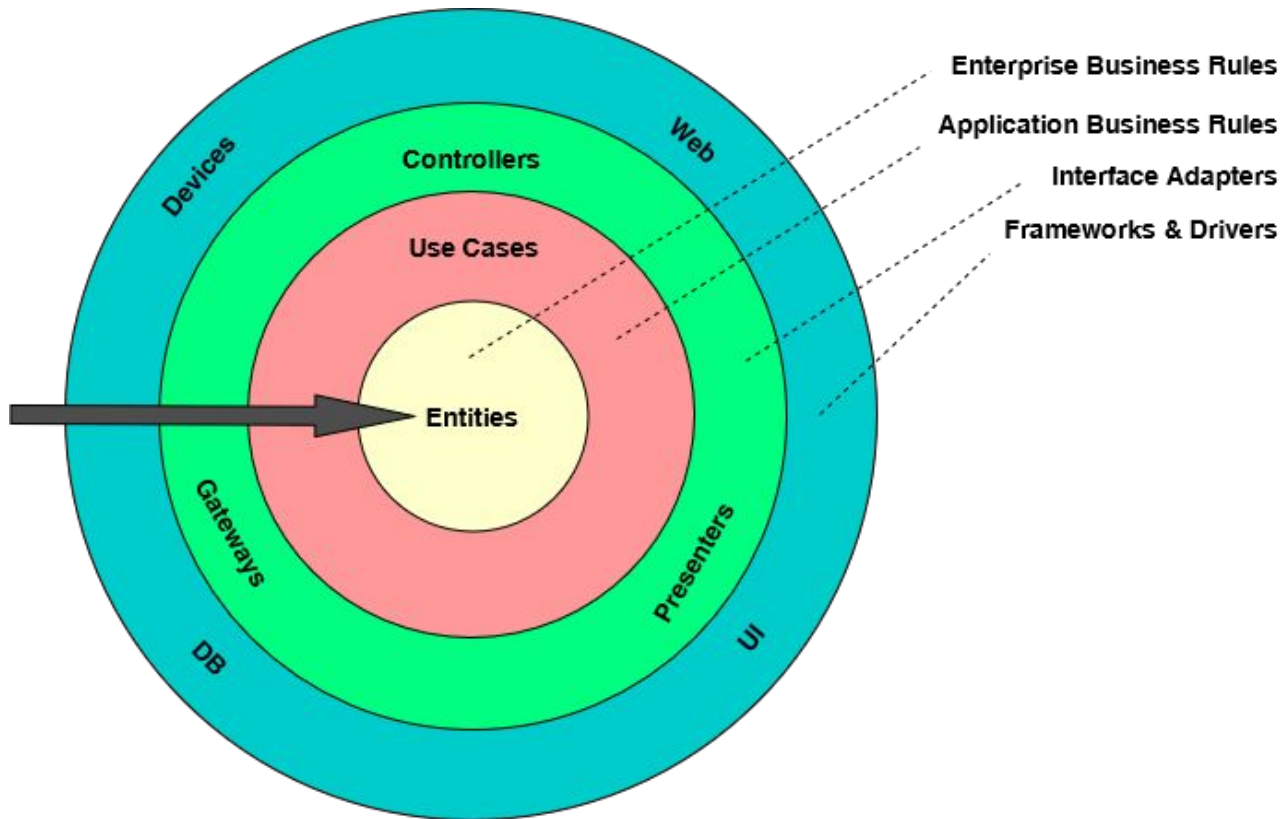
Você pode trocar o banco de dados (PostgreSQL, MongoDB, SQL Server, etc.) sem afetar as regras de negócio.



Clean Architecture

Clean Architecture é um conjunto de princípios de design de software cujo objetivo principal é a separação de preocupações.

A estrutura é visualmente representada por círculos concêntricos, onde as camadas mais internas são as políticas de mais alto nível e as camadas mais externas são os detalhes técnicos de mais baixo nível.



A Regra da Dependência

A regra fundamental que governa essa arquitetura é a **Regra da Dependência**

As dependências do código-fonte só podem apontar para dentro.

Nada em um círculo interno pode saber qualquer coisa sobre um círculo externo.

O código de um círculo interno não pode mencionar o nome de classes, funções ou variáveis declaradas em um círculo externo.



Clean Architecture

As Camadas da Clean Architecture

1. Entidades (Entities)

Esta é a camada mais interna e contém as **Regras Cruciais de Negócios** da empresa.

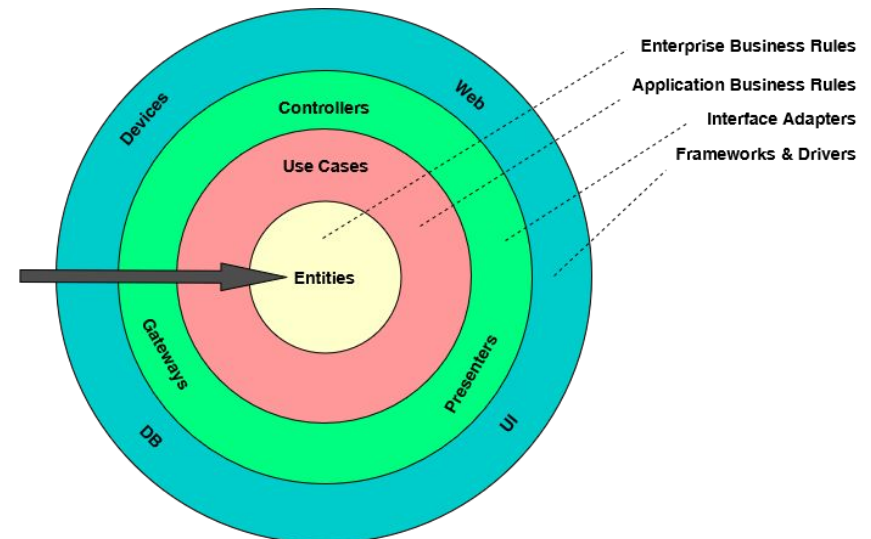
São os objetos que encapsulam as regras mais gerais e de mais alto nível, e são os menos propensos a mudar quando algo externo ao sistema muda.

2. Casos de Uso (Use Cases)

Esta camada contém as **regras de negócio específicas da aplicação**.

Ela orchestra o fluxo de dados para e a partir das entidades para atingir os objetivos de uma funcionalidade do sistema.

As mudanças nesta camada não devem afetar as entidades.



Clean Architecture

As Camadas da Clean Architecture

3. Adaptadores de Interface (Interface Adapters)

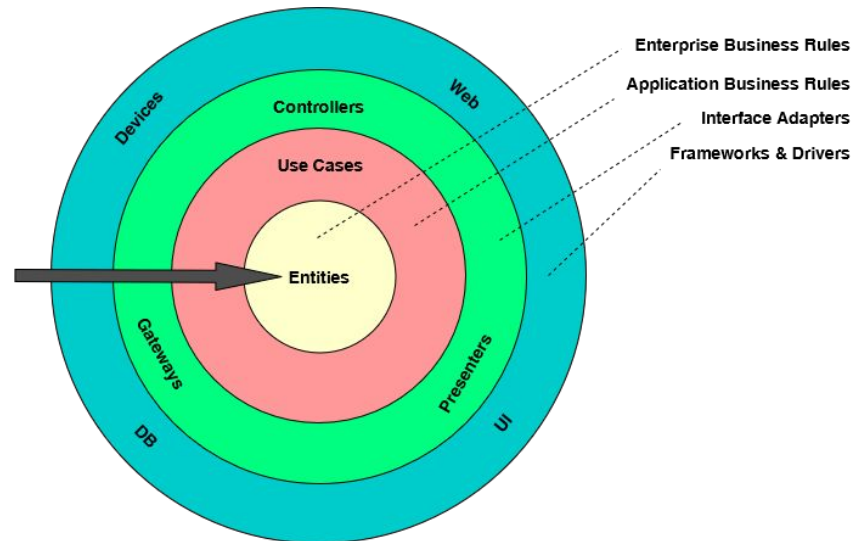
Esta camada é responsável por converter os dados de um formato conveniente para as entidades e casos de uso para um formato conveniente para as camadas externas (como o banco de dados ou a UI), e vice-versa.

É aqui que se encontram as arquiteturas MVC, com seus Controllers, Models e Views.

4. Frameworks e Drivers

A camada mais externa é composta por frameworks e ferramentas, como o banco de dados e o framework web.

Geralmente, o código escrito aqui é mínimo, servindo apenas como "cola" para conectar com a próxima camada interna.



Exemplo Prático PIX

Clean Architecture: Exemplo Prático PIX

Por conta dos diversos aspectos do sistema PIX, iremos focar o exemplo a seguir no evento "Realizar pagamento PIX usando uma chave".

Um cliente, usando o aplicativo do seu banco, digita uma chave PIX, insere um valor e confirma a transação.



Entidades

Clean Architecture: Exemplo Prático PIX

Entidades - O Círculo Mais Interno

Esta camada contém as regras de negócio cruciais, que são a essência do que é o PIX, independentemente de qualquer tecnologia.

O código aqui é puro, sem dependências de frameworks, bancos de dados ou interfaces gráficas.

Nesta camada teríamos objetos como:

TransacaoPIX

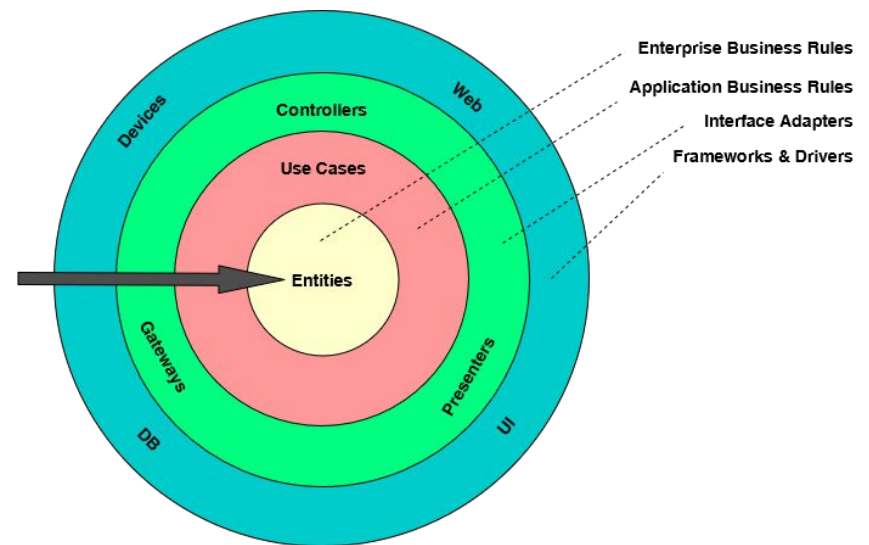
Uma classe que representa a transação em si.

Propriedades

ID da Transação
Conta do Pagador
Conta do Recebedor
Valor
Status (Pendente, Concluída, Falha, Devolvida)
Data/Hora

Métodos (Regras de Negócio)

concluir()
falhar()
devolver(motivo)



Clean Architecture: Exemplo Prático PIX

Entidades - O Círculo Mais Interno

ChavePIX

Representa uma chave PIX.

Propriedades

Tipo da Chave (CPF, CNPJ, E-mail, Celular, Aleatória), Valor da Chave.

Métodos (Regras de Negócio)

validarFormato(), para garantir que um e-mail parece um e-mail ou que um CPF tem o formato correto.

Conta

Representa a conta bancária de um cliente.

Propriedades

ID da Conta, Saldo.

Métodos (Regras de Negócio)

debitar(valor), creditar(valor). O método debitar conteria a lógica para garantir que há saldo suficiente.

LimiteTransacao

Uma entidade que define os limites de valor para transações.

Propriedades

Limite Diurno, Limite Noturno.

Métodos (Regras de Negócio)

verificarLimite(valor, hora)

Casos de Uso

Clean Architecture: Exemplo Prático PIX

Casos de Uso

Esta camada contém a lógica específica da aplicação, **orquestrando as entidades** para realizar uma tarefa.

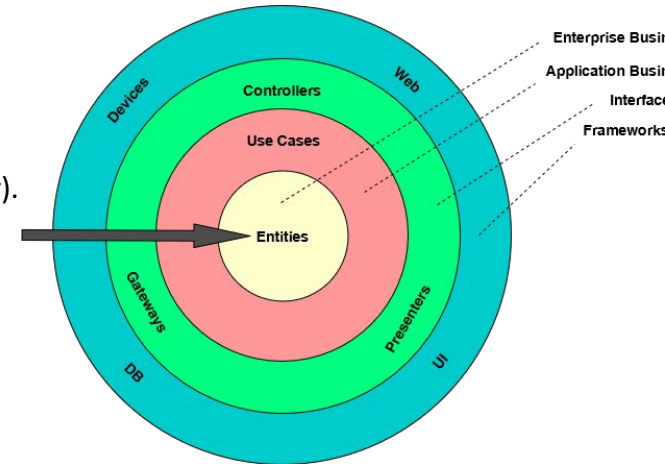
RealizarPagamentoPix:

Entrada

Dados da requisição (ID da conta pagadora, chave PIX do recebedor, valor).

Saída

Dados da resposta (ID da transação, status final, data/hora).



Orquestração:

1. Usa uma interface `IContaRepository` para carregar a Conta do pagador.
2. Usa a entidade `LimiteTransacao` para verificar se o valor está dentro do limite.
3. Usa uma interface `IDiretorioChavesGateway` para buscar os dados da conta do recebedor a partir da chave PIX.
4. Cria uma `TransacaoPIX` com status "Pendente".
5. Chama o método `debitar()` da Conta do pagador.
6. Usa uma interface `IBCGateway` (para se comunicar com o sistema do Banco Central) para efetivar a transferência.
7. Se a comunicação for bem-sucedida, atualiza o status da `TransacaoPIX` para "Concluída" e a salva usando `ITransacaoRepository`.

Outros casos de uso nesta camada seriam

CadastrarChavePix

ConsultarExtratoPix

GerarQrCodeEstatico

Clean Architecture: Exemplo Prático PIX

Caso de Uso: CadastrarChavePix

Entrada

Um objeto simples contendo idDaConta, tipoDeChave (ex: "EMAIL") e valorDaChave (ex: "cliente@email.com").

Saída

Um objeto simples indicando sucesso ou falha, com o idDaChave criada e a dataDeCriacao.

Orquestração

1. Instancia uma entidade ChavePIX com os dados recebidos.
2. Chama o método de negócio da entidade chave.validarFormato() para garantir que a chave é válida (ex: o e-mail tem um formato de e-mail). Se for inválida, retorna um erro.
3. Utiliza uma interface de gateway (ex: IDictGateway) para consultar o DICT (Diretório de Identificadores de Contas Transacionais do Banco Central) e verificar se aquela chave já está em uso. Se estiver, retorna um erro.
4. Utiliza uma interface de repositório (ex: IContaRepository) para carregar a entidade Conta do cliente.
5. Associa a ChavePIX à Conta.
6. Utiliza outra interface de repositório (ex: IChavePixRepository) para salvar a nova chave no banco de dados interno do banco PIX.
7. Chama o IDictGateway novamente para registrar oficialmente a chave no DICT.
8. Retorna o objeto de resposta com sucesso.

Clean Architecture: Exemplo Prático PIX

Caso de Uso: Consultar Extrato PIX

Permitir que um cliente visualize todas as suas transações PIX (enviadas e recebidas) em um determinado período.

Entrada

Um objeto simples com idConta, dataInicio e dataFim.

Saída

Uma lista de objetos simples, cada um contendo dados de uma transação (valor, data, tipo, nome da contraparte, status).

Orquestração

1. Recebe o objeto de requisição com o período desejado.
2. Utiliza uma interface de repositório (ex: ITransacaoPixRepository) e chama o método buscarPorContaEPeriodo(idDaConta, dataInicio, dataFim).
3. O repositório retorna uma lista de entidades TransacaoPIX.
4. O caso de uso percorre a lista de entidades e a transforma em uma lista de objetos de resposta simples e formatados, prontos para serem exibidos.
5. Retorna a lista de transações formatada.

Controllers, Presenters e Gateways

Clean Architecture: Exemplo Prático PIX

Controllers, Presenters e Gateways (adaptadores)

- Principal propósito é adaptar ou converter dados. Pense nesta camada como a "zona de tradução" do seu sistema.
- Fica entre o núcleo de negócio (Entidades e Casos de Uso) e o mundo exterior (Frameworks e Drivers).
- Sua função é converter os dados do formato mais conveniente para o núcleo do negócio para o formato mais conveniente para as ferramentas externas, e vice-versa.

Exemplo 1: O Controller (Adaptador de Entrada para a Web)

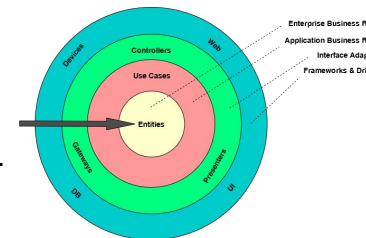
Controller pertence a esta camada porque ele adapta os detalhes do protocolo HTTP para a linguagem do seu domínio de negócio.

Exemplo 2: O Presenter (Adaptador de Saída para a UI)

Adapta os dados do domínio para o formato de apresentação exigido pela interface do usuário.

Exemplo 3: O Repositório (Adaptador para o Banco de Dados)

Converte as necessidades do domínio em comandos específicos para a tecnologia de persistência, isolando o núcleo de qualquer conhecimento sobre o banco de dados.



Clean Architecture: Exemplo Prático PIX

Controllers, Presenters e Gateways (adaptadores)

O Controller (Adaptador de Entrada para a Web)

Imagine que o cliente clica em "Pagar" no aplicativo do banco. O aplicativo envia uma requisição HTTP para a API do seu sistema.

O Mundo Externo (Framework)

O framework web (como o Spring Boot) recebe uma requisição HTTP crua.

Essa requisição contém dados em um formato específico da web: um corpo em JSON, cabeçalhos de autenticação, parâmetros na URL, etc.

O Núcleo do Negócio (Caso de Uso)

O seu caso de uso RealizarPagamentoPix não sabe o que é JSON ou HTTP.

Ele espera receber um objeto Java/Python/C# simples e limpo (um Request Model) com os dados necessários: idContaPagador, chaveDestino, valor.

Clean Architecture: Exemplo Prático PIX

Controllers, Presenters e Gateways (adaptadores)

O Presenter (Adaptador de Saída para a UI)

Após o caso de uso RealizarPagamentoPix terminar, ele retorna um objeto de resposta simples (Response Model) com dados puros BigDecimal para o valor, LocalDateTime para a data, e um ENUM para o status.

O Núcleo do Negócio (Caso de Uso)

Retorna dados brutos, sem formatação.

O Mundo Externo (UI/View)

A tela do aplicativo precisa exibir esses dados de forma amigável para o usuário

Valor formatado como moeda ("R\$ 50,00"), a data como texto ("29 de setembro de 2025, 14h35") e o status em uma cor específica (verde para "Concluído").

O Papel do PixPresenter (O Adaptador)

Recebe os Dados Puros

O PixPresenter recebe o objeto de resposta do caso de uso.

Formata para Apresentação

Converte o BigDecimal para uma string "R\$ 50,00". Converte a data para "29/09/2025 14h35". Define uma flag booleana mostrarIconeDeSucesso = true.

Cria o ViewModel

Agrupar todos esses dados formatados em um novo objeto simples chamado ViewModel.

Entrega para a View

A View (a tela do app) recebe o ViewModel e sua única tarefa é exibir os dados que já estão prontos, sem nenhuma lógica de formatação.

Clean Architecture: Exemplo Prático PIX

Controllers, Presenters e Gateways (adaptadores)

O Repositório (Adaptador para o Banco de Dados)

O caso de uso RealizarPagamentoPix precisa buscar os dados da conta do cliente e salvar a transação final.

O Núcleo do Negócio (Caso de Uso)

Ele não sabe o que é SQL. Ele simplesmente invoca um método de uma interface que ele mesmo define, como `IContaRepository.buscarPorId("123")`. Ele espera receber de volta um objeto de entidade `Conta`.

O Mundo Externo (Driver)

O banco de dados PostgreSQL, que entende apenas a linguagem SQL e retorna dados em um formato de linhas e colunas (`ResultSet`).

O Papel do `ContaRepositorySQL` (O Adaptador)

Implementa a Interface

Implementa a interface `IContaRepository` definida na camada de Casos de Uso.

Traduz a Chamada

Quando o método `buscarPorId("123")` é chamado, o repositório o traduz para um comando SQL: `SELECT * FROM contas WHERE id = '123'`.

Executa no Mundo Externo

Usa o driver do banco de dados para executar a query.

Converte a Resposta

Pega o resultado em formato de tabela (o `ResultSet`) e o converte de volta para uma instância da entidade `Conta`, que é o que o caso de uso espera.

Clean Architecture: Exemplo Prático PIX

Em resumo, **esta camada (Controllers, Presenters e Gateways)** adapta a tecnologia do mundo exterior para um formato que os **Casos de Uso e Entidades (camadas internas)** entendem, e vice-versa. Exemplos:

PixController

Recebe requisições HTTP do aplicativo móvel do banco. Ele não tem lógica de negócio; apenas extrai os dados da requisição, passa para o Caso de Uso RealizarPagamentoPix e retorna a resposta.

PixPresenter

Formata os dados de saída do Caso de Uso em um ViewModel, uma estrutura de dados simples para ser exibida na tela do celular do cliente.

ContaRepositorySQL

Uma implementação da interface IContaRepository. Esta classe contém o código SQL para buscar e atualizar os dados de uma Conta no banco de dados do banco.

SpiGatewayRest

Uma implementação da interface ISpiGateway. Contém o código que faz as chamadas de rede (ex: REST) para o SPI (Sistema de Pagamentos Instantâneos) do Banco Central, tratando da autenticação e dos formatos de mensagem específicos.

Frameworks e Drivers

Clean Architecture: Exemplo Prático PIX

Esta é a camada dos detalhes, onde as tecnologias vivem.

Framework Web

Spring Boot, que recebe as requisições HTTP e as direciona para o PixController.

Banco de Dados

PostgreSQL, onde as informações das contas e transações são fisicamente armazenadas.

Interface do Usuário (UI)

O aplicativo do banco para iOS/Android, que envia a requisição de pagamento para o PixController.

Dispositivos Externos

O sistema de registros do Banco Central, com o qual o SpiGatewayRest se comunica.

Clean Architecture: Exemplo Prático PIX

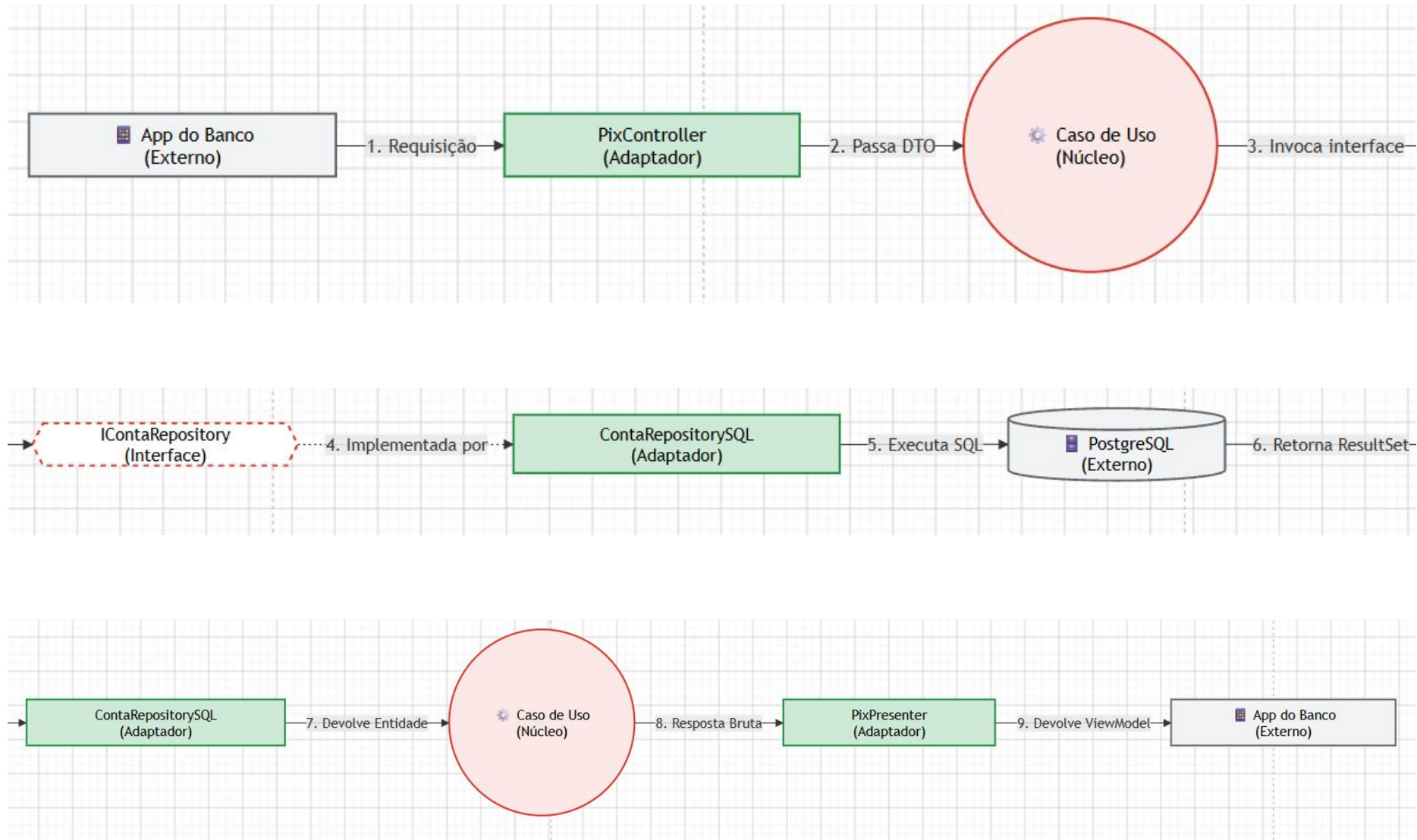
O Fluxo e a Regra da Dependência

1. O App do Banco (UI) envia uma requisição.
2. O Spring Boot (Framework) recebe e direciona ao PixController (Adaptador de Entrada).
3. O PixController aciona o Caso de Uso RealizarPagamentoPix passando os dados.
4. O Caso de Uso orquestra as Entidades (Conta, TransacaoPIX). Para falar com o banco, ele não chama o banco, ele chama interfaces que ele mesmo define (ex: IContaRepository).
5. Em tempo de execução, a chamada da interface é atendida pelos Adaptadores de Saída (ContaRepositorySQL, SpiGatewayRest), que traduzem a chamada para SQL ou REST usando os Drivers/Frameworks.
6. Ao finalizar, o Caso de Uso devolve os dados brutos, que passam pelo PixPresenter (Adaptador) para serem formatados e enviados de volta à UI.

Observe o passo 4 e 5: O fluxo de execução sai do Caso de Uso em direção ao Banco de Dados, mas a dependência do código aponta para dentro. O Caso de Uso depende apenas da interface IContaRepository. É o adaptador ContaRepositorySQL que depende dessa interface para implementá-la.

Isso torna o coração do sistema (a lógica do PIX) imune a mudanças de tecnologia.

Clean Architecture: Exemplo Prático PIX



Clean Architecture: Outro exemplo prático

Você pode copiar o código abaixo e colar no **draw.io** (via [Organizar](#) > [Inserir](#) > [Avançado](#) > [Mermaid](#)):

```
flowchart LR
    subgraph Externa [Camada 4: Frameworks & Drivers]
        direction TB
        API["🌐 Rota Web / API\n(Express, Spring Web)"]
        Banco["🏦 Banco de Dados\n(PostgreSQL, Prisma)"]
    end

    subgraph Adaptadores [Camada 3: Adapters]
        direction TB
        Ctrl["PedidoController\n(Traduz HTTP para DTO)"]
        RepoImpl["PedidoRepositoryImpl\n(Traduz Entidade para SQL)"]
    end

    subgraph CasosDeUso [Camada 2: Use Cases]
        direction TB
        UC["🔗 FinalizarCompraUseCase\n(Orquestra a regra de negócio)"]
        IRepo["IPedidoRepository\n(Interface / Contrato)"]
    end

    subgraph Entidades [Camada 1: Entities]
        direction TB
        Pedido["📦 Entidade Pedido\n(Regras puras: cálculo de total, frete)"]
    end

    %% Fluxo de Execução (Linhas Sólidas Grossas)
    API == "1. Requisição HTTP" ==> Ctrl
    Ctrl == "2. Passa Dados (DTO)" ==> UC
    UC == "3. Aplica regras" ==> Pedido
    UC == "4. Solicita salvamento" ==> IRepo
    RepoImpl == "5. Executa Query" ==> Banco

    %% Regra de Dependência (Linhas Tracejadas Vermelhas)
    Ctrl -. "Importa" .-> UC
    UC -. "Importa" .-> Pedido
    RepoImpl -. "Implementa\n(Inversão de Dependência)" .-> IRepo

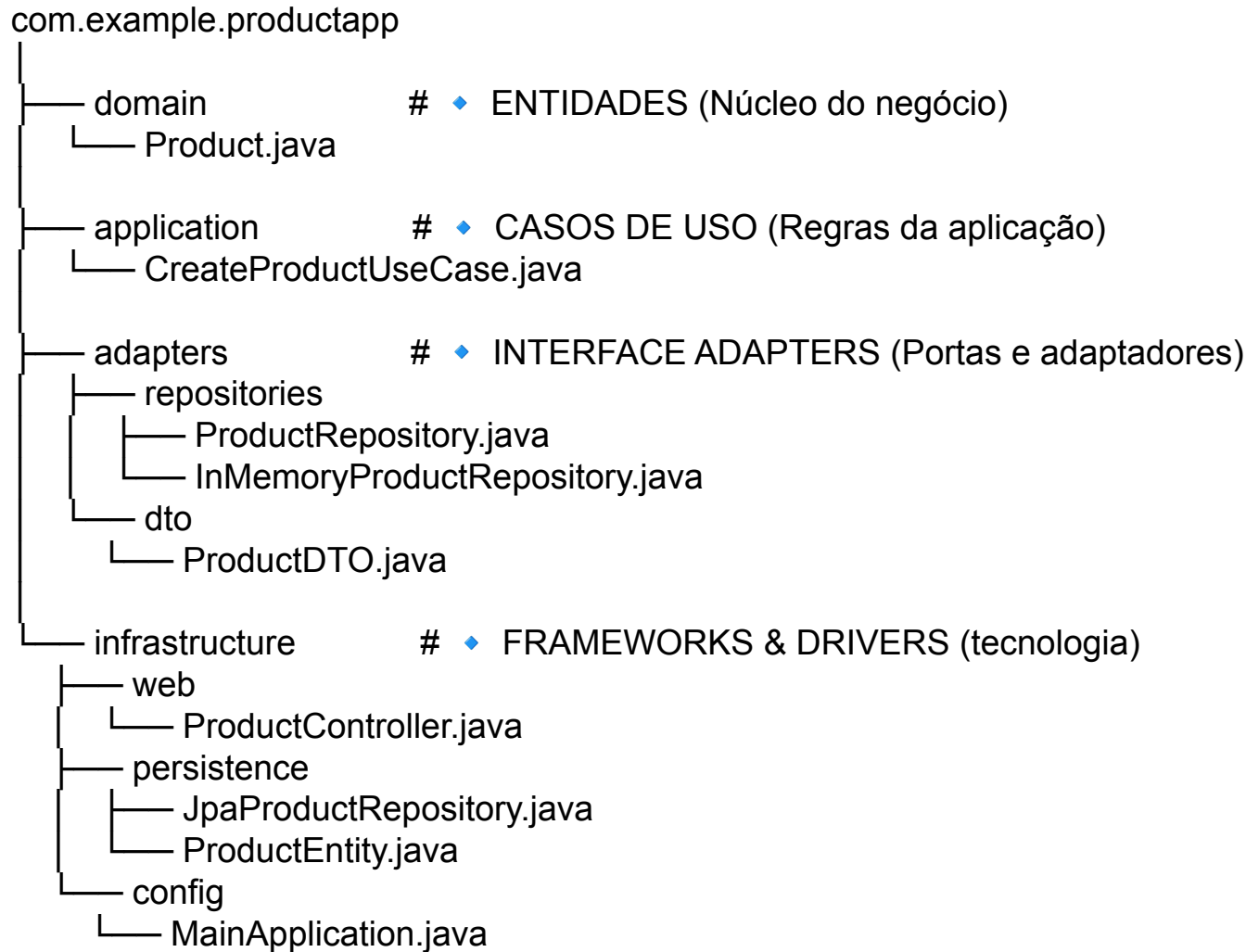
    %% Estilização para as camadas
    classDef c4 fill:#e3f2fd,stroke:#1565c0,stroke-width:2px,color:#000
    classDef c3 fill:#e8f5e9,stroke:#2e7d32,stroke-width:2px,color:#000
    classDef c2 fill:#fff3e0,stroke:#e65100,stroke-width:2px,color:#000
    classDef c1 fill:#ffebee,stroke:#c62828,stroke-width:2px,color:#000

    class API,Banco c4
    class Ctrl,RepoImpl c3
    class UC,IRepo c2
    class Pedido c1

    linkStyle 5,6,7 stroke:#d32f2f,stroke-width:2px,stroke-dasharray: 5 5;
```

Exemplo Simples em Código

Clean Architecture: Exemplo Código



Clean Architecture: Exemplo Código

```
// domain/Product.java
package com.example.productapp.domain;

public class Product {
    private final String id;
    private final String name;
    private final double price;

    public Product(String id, String name, double price) {
        if (price < 0) throw new IllegalArgumentException("Preço inválido");
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public double getPrice() { return price; }
}
```

Entidades

Camada mais
interna (Nível 1)

Clean Architecture: Exemplo Código

```
// application/CreateProductUseCase.java
package com.example.productapp.application;

import com.example.productapp.domain.Product;
import com.example.productapp.adapters.repositories.ProductRepository;
import java.util.UUID;

public class CreateProductUseCase {

    private final ProductRepository repository;

    public CreateProductUseCase(ProductRepository repository) {
        this.repository = repository;
    }

    public Product execute(String name, double price) {
        Product product = new Product(UUID.randomUUID().toString(), name,
price);
        repository.save(product);
        return product;
    }
}
```

Caso de Uso
(regra da aplicação)

Camada Nível 2

Clean Architecture: Exemplo Código

```
// adapters/repositories/ProductRepository.java
package com.example.productapp.adapters.repositories;

import com.example.productapp.domain.Product;

public interface ProductRepository {
    void save(Product product);
}
```

Interface
(contrato do repositório)

```
// adapters/dto/ProductDTO.java
package
com.example.productapp.adapters.dto;

public record ProductDTO(String name,
double price) {}
```

Camada Nível 3

Clean Architecture: Exemplo Código

```
// infrastructure/persistence/JpaProductRepository.java
package com.example.productapp.infrastructure.persistence;

import com.example.productapp.domain.Product;
import com.example.productapp.adapters.repositories.ProductRepository;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import org.springframework.stereotype.Repository;

@Repository
public class JpaProductRepository implements ProductRepository {

    @PersistenceContext
    private EntityManager em;

    @Override
    public void save(Product product) {
        ProductEntity entity = new ProductEntity(
            product.getId(), product.getName(), product.getPrice()
        );
        em.persist(entity);
    }
}
```

Implementação concreta do
Repositório (usando JPA)

Framework/Driver
(entrada do sistema)

Camada Nível 4
(mais externa)

Clean Architecture: Exemplo Código

```
// infrastructure/web/ProductController.java
package com.example.productapp.infrastructure.web;

import com.example.productapp.application.CreateProductUseCase;
import com.example.productapp.adapters.dto.ProductDTO;
import com.example.productapp.domain.Product;
import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;

@RestController
@RequestMapping("/products")
public class ProductController {

    private final CreateProductUseCase createProductUseCase;

    public ProductController(CreateProductUseCase useCase) {
        this.createProductUseCase = useCase;
    }

    @PostMapping
    public ResponseEntity<Product> create(@RequestBody ProductDTO dto) {
        Product created = createProductUseCase.execute(dto.name(), dto.price());
        return ResponseEntity.ok(created);
    }
}
```

Controller REST (Entrada HTTP)

**Framework/Driver
(entrada do sistema)**

Camada Nível 4
(mais externa)

Clean Architecture: Exemplo Código

```
// infrastructure/config/MainApplication.java
package com.example.productapp.infrastructure.config;
```

Ponto de entrada (Main Application)

```
import com.example.productapp.adapters.repositories.ProductRepository;
import com.example.productapp.application.CreateProductUseCase;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
```

Framework/Driver
(entrada do sistema)

```
@SpringBootApplication(scanBasePackages = "com.example.productapp")
public class MainApplication {
```

```
    public static void main(String[] args) {
        SpringApplication.run(MainApplication.class, args);
    }
```

```
    @Bean
    public CreateProductUseCase createProductUseCase(ProductRepository repository) {
        return new CreateProductUseCase(repository);
    }
}
```

Camada Nível 4
(mais externa)

Dúvidas?



www.shutterstock.com · 744867163

Service-Based Architecture

Service-Based Architecture

- Service-Based vs Microserviços
 - Similar, mas diferente!
- Ambas envolvem a criação de sistemas compostos por serviços, mas divergem principalmente em
 - Granularidade
 - Acoplamento
 - Escalabilidade
 - Comunicação

- Microserviços:
 - Cada serviço é extremamente pequeno e realiza uma única função ou responsabilidade (princípio do Single Responsibility).
 - São projetados para serem altamente especializados e autônomos.
 - Exemplos
 - Um serviço apenas para autenticação por email e senha, outro para login social, outro para pagamento por cartão de crédito, etc.
- Service-Based
 - Os serviços são maiores e abrangem múltiplas responsabilidades relacionadas a um domínio específico.
 - Um único serviço pode conter diversas funcionalidades relacionadas.
 - Exemplos: um serviço completo de "Gestão de Clientes" que inclui autenticação, perfil e histórico de compras.

- Microserviços:
 - Princípio da Responsabilidade Única (SRP)
 - Totalmente independentes em termos de desenvolvimento, implantação e manutenção.
 - Podem ser desenvolvidos e implantados em diferentes linguagens de programação ou plataformas.
 - Comunicação entre microserviços geralmente ocorre via APIs REST ou mensagens assíncronas.
- Service-Based:
 - Os serviços têm menor nível de independência.
 - Geralmente, são parte de uma implantação monolítica maior, ainda que sejam logicamente separados.
 - A comunicação entre serviços pode ser mais acoplada e frequentemente dentro do mesmo ambiente de execução.
 - Geralmente usada como transição entre monolítico modular e microserviços.

- Microserviços:
 - Escalabilidade granular
 - apenas os serviços que exigem mais recursos são escalados.
 - Ideal para lidar com diferentes taxas de carga em partes específicas do sistema.
- Service-Based:
 - Escalabilidade mais limitada, já que os serviços costumam ser mais monolíticos e menos independentes.
 - Escalar um serviço pode exigir escalar toda a aplicação ou partes relacionadas.

- Microserviços:
 - Comunicação é mais complexa, baseada em APIs REST ou mensagens assíncronas
 - Ex.: RabbitMQ e Kafka
 - Projetados para funcionarem mesmo em ambientes distribuídos (diferentes servidores ou nuvens).
- Service-Based
 - Comunicação tende a ser mais direta e dentro de uma infraestrutura comum.
 - Geralmente é síncronica
 - chamadas de procedimento remoto ou invocações internas.

Service-Based Architecture

Aspecto	Microserviços	Service-Based
Granularidade	♦ Fina — serviços pequenos e especializados	♦ Grossa — serviços maiores, com várias responsabilidades
Independência	🧩 Totalmente independentes (deploy isolado)	🔗 Interdependentes, geralmente implantados juntos
Escalabilidade	📈 Escalável de forma granular (por serviço)	📊 Escalável, mas depende de serviços associados
Comunicação	🌐 REST APIs / mensagens assíncronas (Kafka, RabbitMQ)	🔄 Comunicação local e síncrona (chamadas internas)
Complexidade Operacional	⚡ Alta — requer orquestração e observabilidade	🧱 Moderada — mais simples de gerenciar
Exemplos de Uso	🚀 Netflix, Uber, Amazon	🏢 Sistemas corporativos internos, ERPs médios

Service-Based vs Microserviços

Imagine um sistema de e-commerce tradicional dividido em serviços grandes por domínio.

Serviço	Responsabilidades
Serviço de Clientes	Cadastro, autenticação, atualização de perfil, histórico de compras
Serviço de Produtos	Cadastro de produtos, controle de estoque, categorias
Serviço de Vendas	Carrinho, processamento de pedidos, emissão de notas
Serviço de Pagamentos	Processamento de cartão, boleto, integração com gateways

Microserviço	Responsabilidade única
Auth-Service	Login, logout, tokens JWT
User-Profile-Service	Dados do usuário, endereço, preferências
Product-Catalog-Service	Lista de produtos e categorias
Inventory-Service	Controle de estoque
Order-Service	Criação e rastreamento de pedidos
Payment-Service	Processamento de pagamentos
Notification-Service	Envio de emails e SMS
Review-Service	Avaliações de produtos

Service-Based Architecture

- A Arquitetura Baseada em Serviços pode ser “quebrada” de várias formas, pode-se separar por
 - Domínio ou funcionalidades
 - Interfaces do usuário
 - como o usuário interage com o serviços
 - Bancos de dados
 - etc

Service-Based Architecture

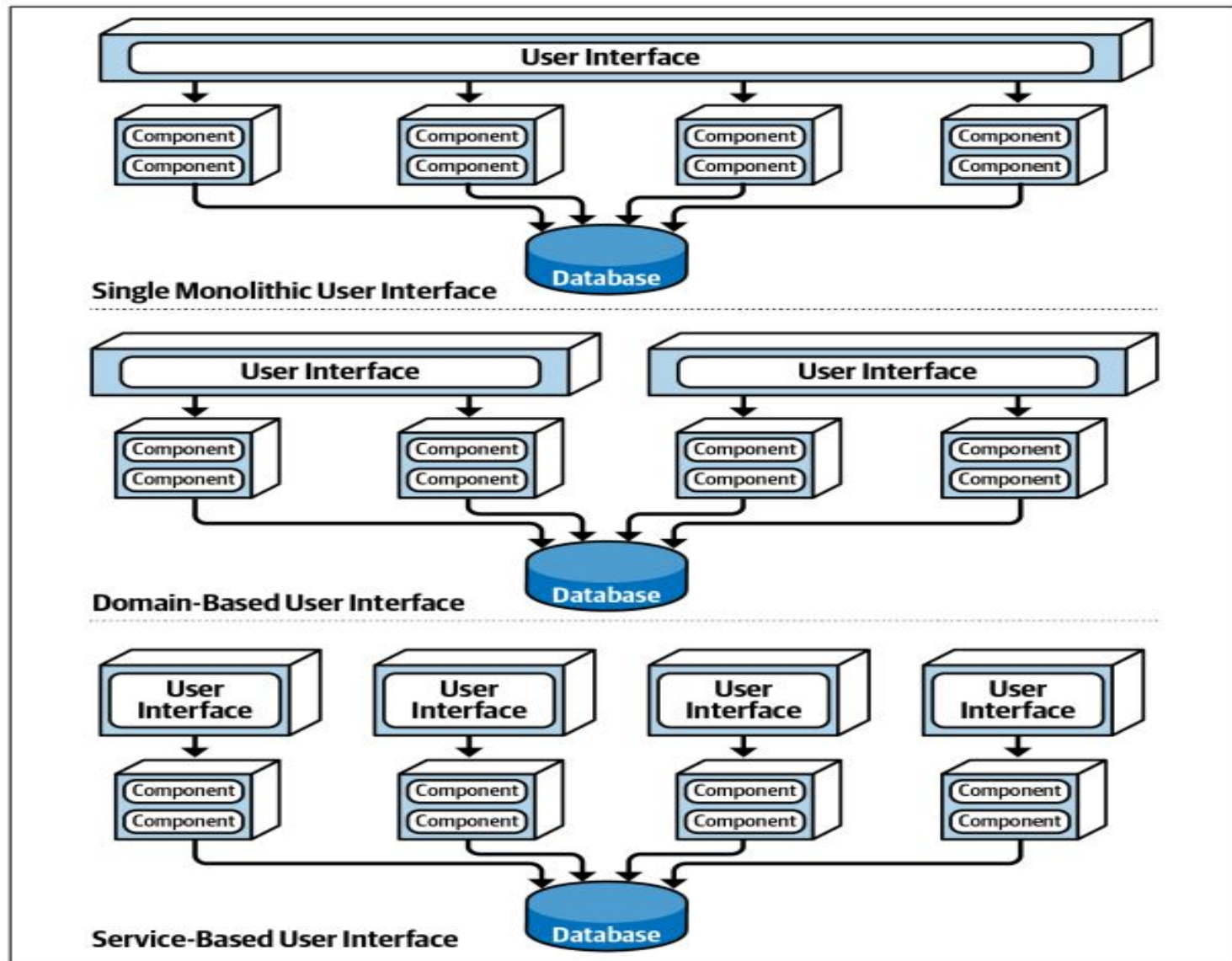


Figure 13-2. User interface variants

Service-Based Architecture

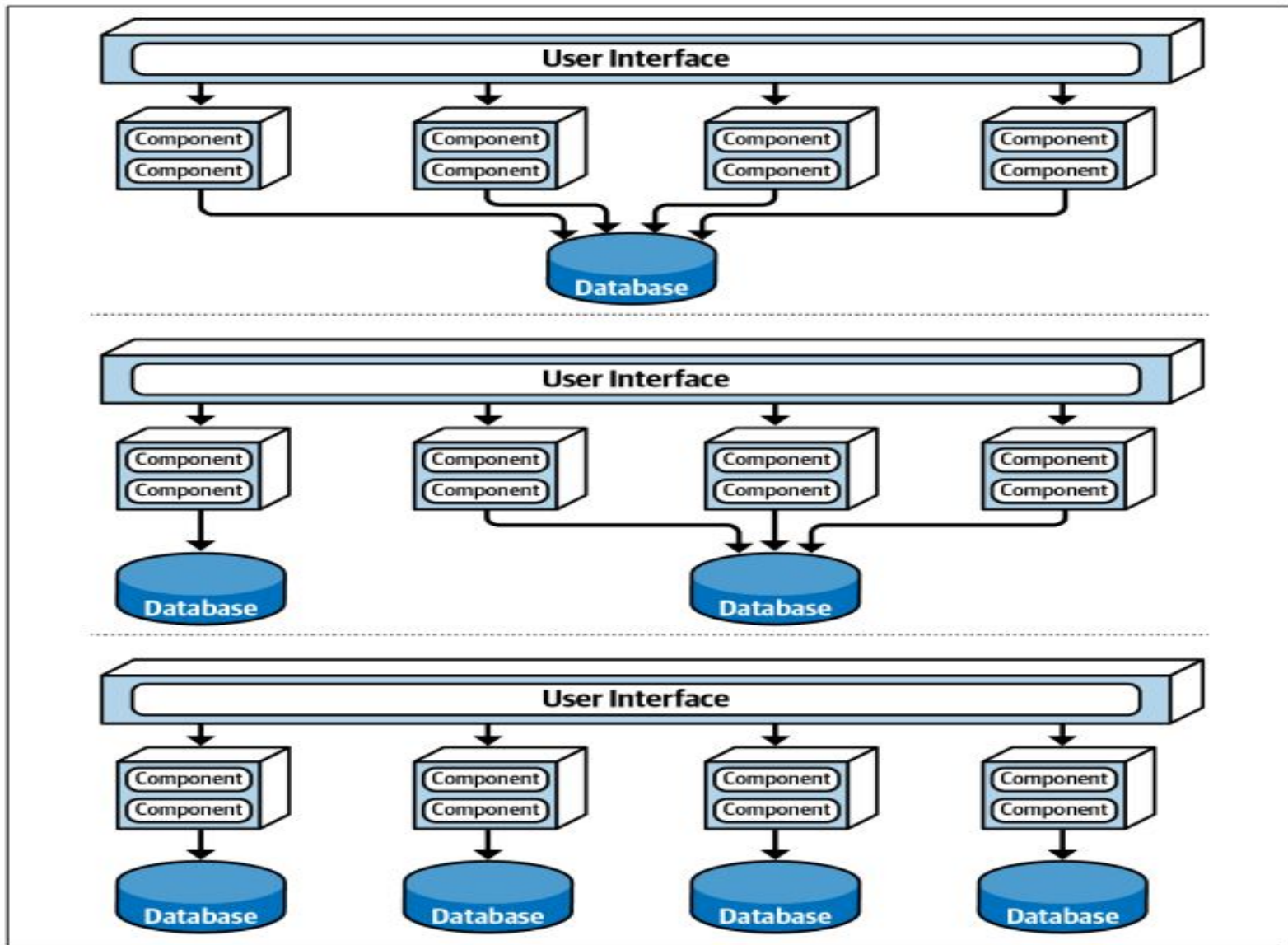


Figure 13-3. Database variants

Service-Based Architecture

- Para não permitir que o usuário acesse diretamente a camada de negócio dos serviços, ou seja, acesse diretamente o serviço, pode-se incluir uma camada de *Gateway*

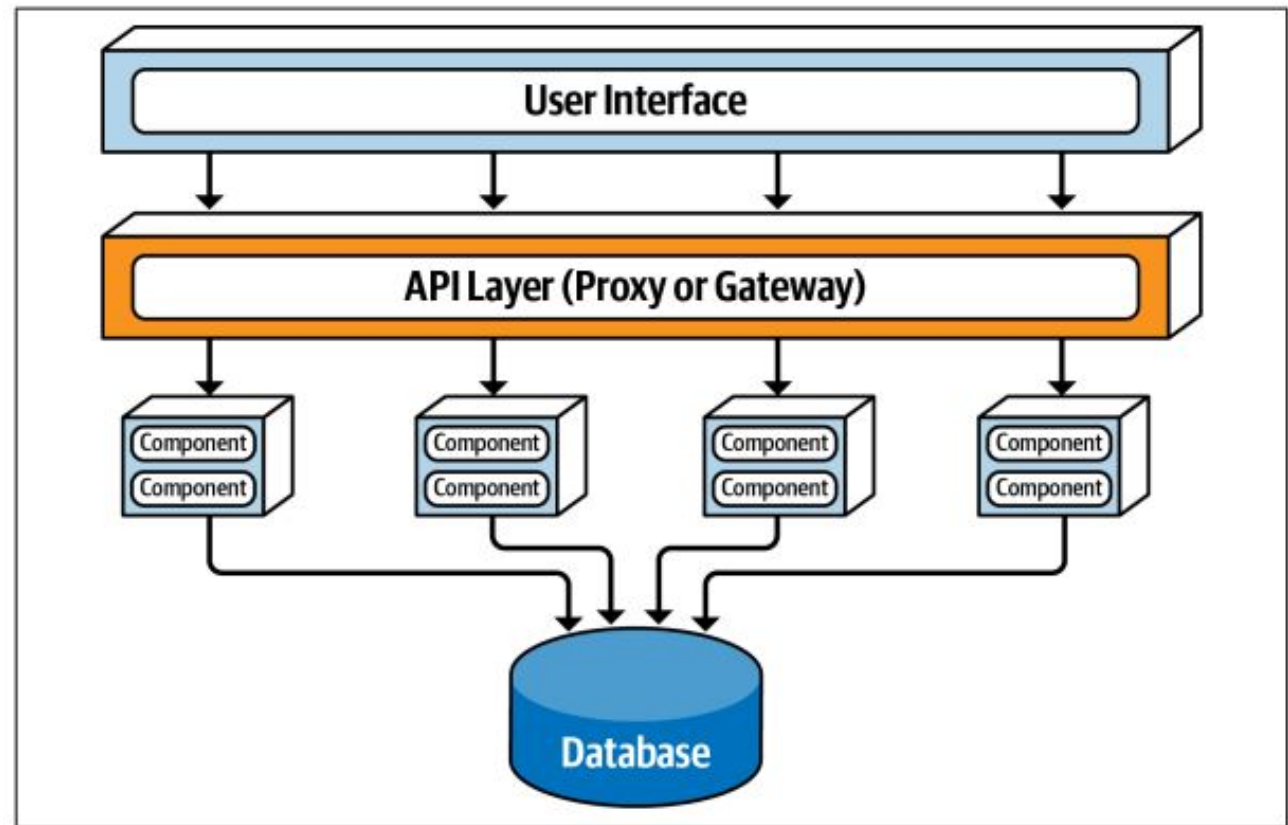
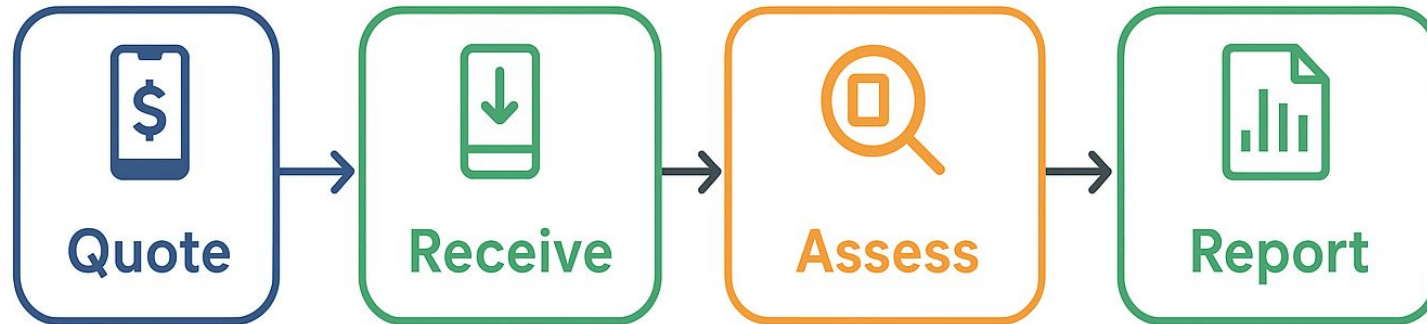


Figure 13-4. Adding an API layer between the user interface and domain services

Service-Based Architecture

- Sistema para reciclagem de Smartphones
 - Usuário requisita um orçamento usando o website ou uma máquina de autoatendimento
 - O processo de *quoting* irá definir quanto vale o smartphone do usuário
 - Se usuário aceitar a proposta de valor a receber, o usuário deve enviar o smartphone para a empresa (processo de *receiving*)
 - Ao receber o smartphone, a empresa irá decidir se está operável ou se será descartado (processo de *assessment*).
 - Se o dispositivo está como esperado, a empresa paga ao usuário (processo de *accounting*).
 - O usuário pode ir ao website da empresa a qualquer momento e verificar o status da reciclagem (processo de *item status*).
 - A partir do resultado do processo de *assessment*, o dispositivo é destruído ou revendido (processo de *recycling*).
 - Por fim, a empresa executa periodicamente a geração de relatórios para as atividades de reciclagem (processo de *reporting*).

Service-Based Architecture



Service-Based Architecture

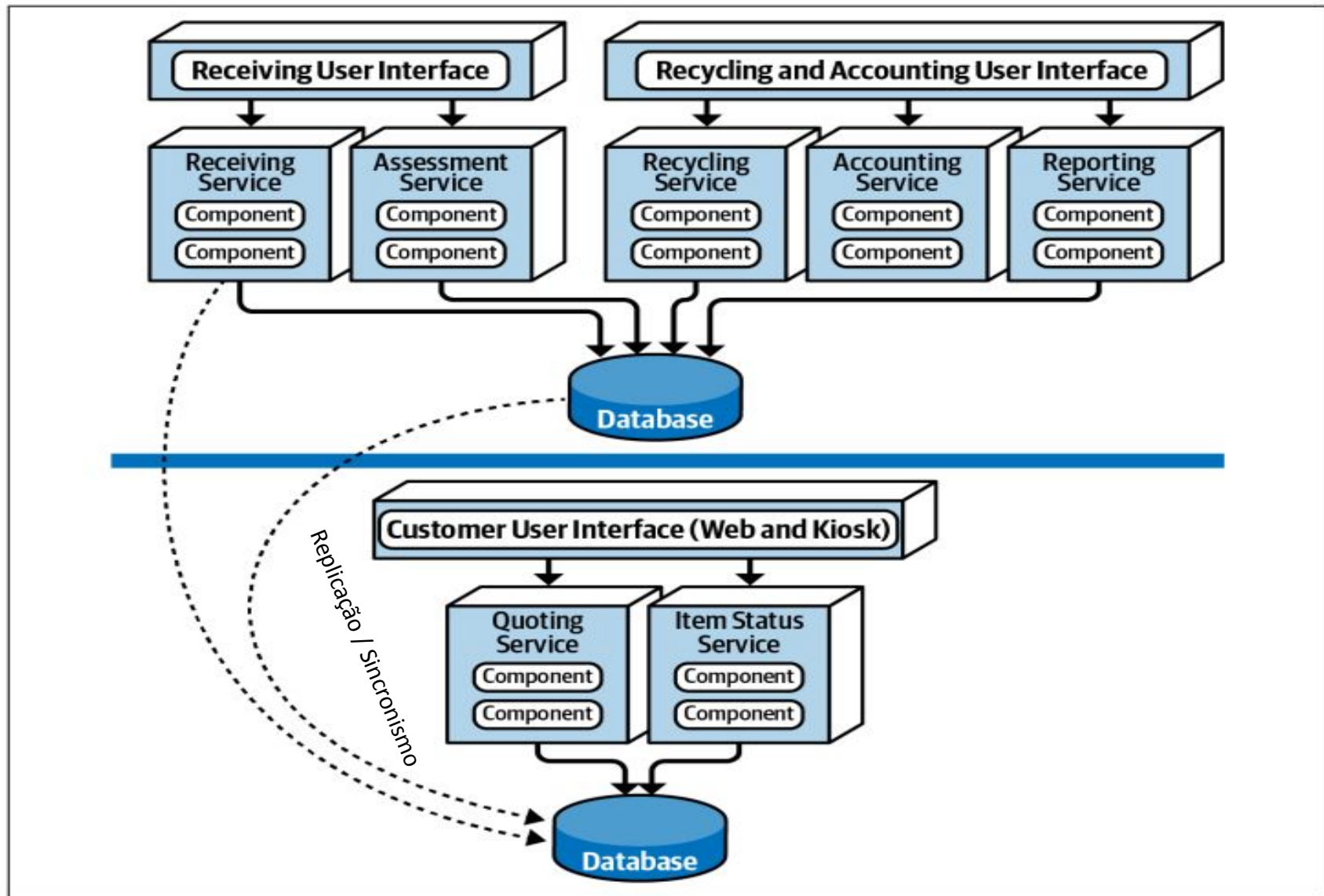


Figure 13-8. Electronics recycling example using service-based architecture

Service-Based Architecture

- Atente a separação de cada “domínio” como um módulo / serviço do sistema.
- Alguns módulos / serviços podem ser escalados de acordo com a necessidade, sem precisar fazer o *deploy* de todo o sistema.
 - Como nas arquiteturas monolíticas...

Service-Based Architecture

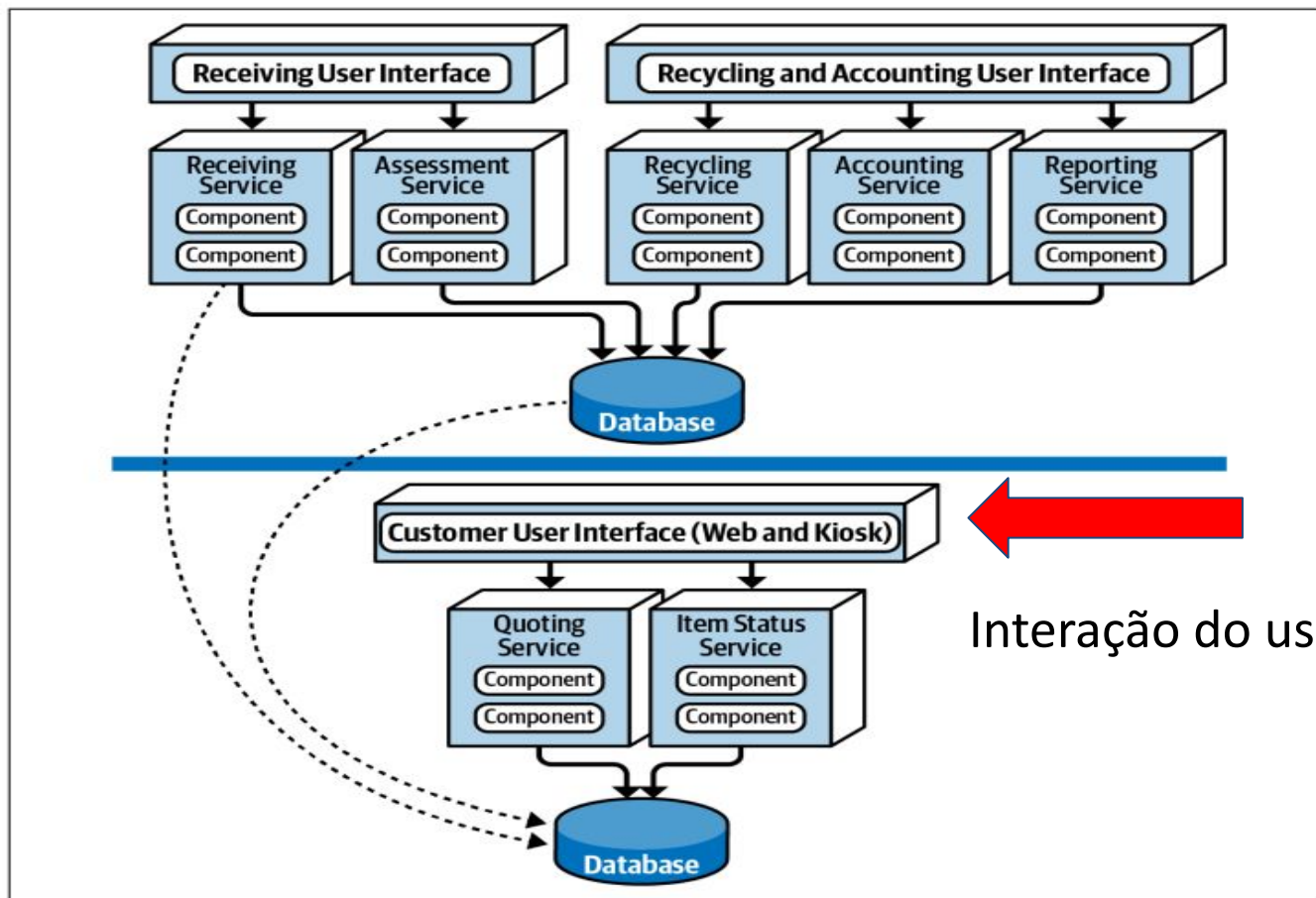


Figure 13-8. Electronics recycling example using service-based architecture

O usuário não precisa conhecer a interface dos demais serviços (internos).
Quais atributos de qualidade podem ser impactados por essa decisão?

Service-Based Architecture

- Escalabilidade
 - É possível escalar serviços distintos
- Tolerância a falhas
 - Se um serviço falhar é mais fácil reparar ou substituir
- Segurança
 - Usuário não conhece procedimentos internos dos serviços

Service-Based Architecture

- Processo de *assessment* é alterado constantemente para adicionar novas regras de negócios e novos produtos que podem ser recebidos.
- Essa alteração frequente é isolada do restante do sistema, provendo
 - Agilidade, ou seja, resposta rápida as mudanças
 - Testabilidade, ou seja, é mais fácil testar
 - *Deployability*, ou seja, reduzindo os riscos associados a um *deploy*. Se é implantado somente um serviço é mais simples e menos perigoso.

Service-Based Architecture

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★
Evolutionary	★★★
Fault tolerance	★★★★
Modularity	★★★★
Overall cost	★★★★
Performance	★★★
Reliability	★★★★
Scalability	★★★
Simplicity	★★★
Testability	★★★★

Service-Based Architecture

- Como uma arquitetura distribuída, o *quanta* pode ser maior ou igual a 1.
- Porém, se houver 10 ou mais serviços, mas que compartilham o mesmo banco de dados, não haverá “elementos isolados” e o *quanta* será 1
 - *Quanta* é a menor unidade lógica ou funcional de um sistema que pode ser escalada ou distribuída independentemente
- A seguir um exemplo de outra modelagem para o sistema em análise...

Service-Based Architecture

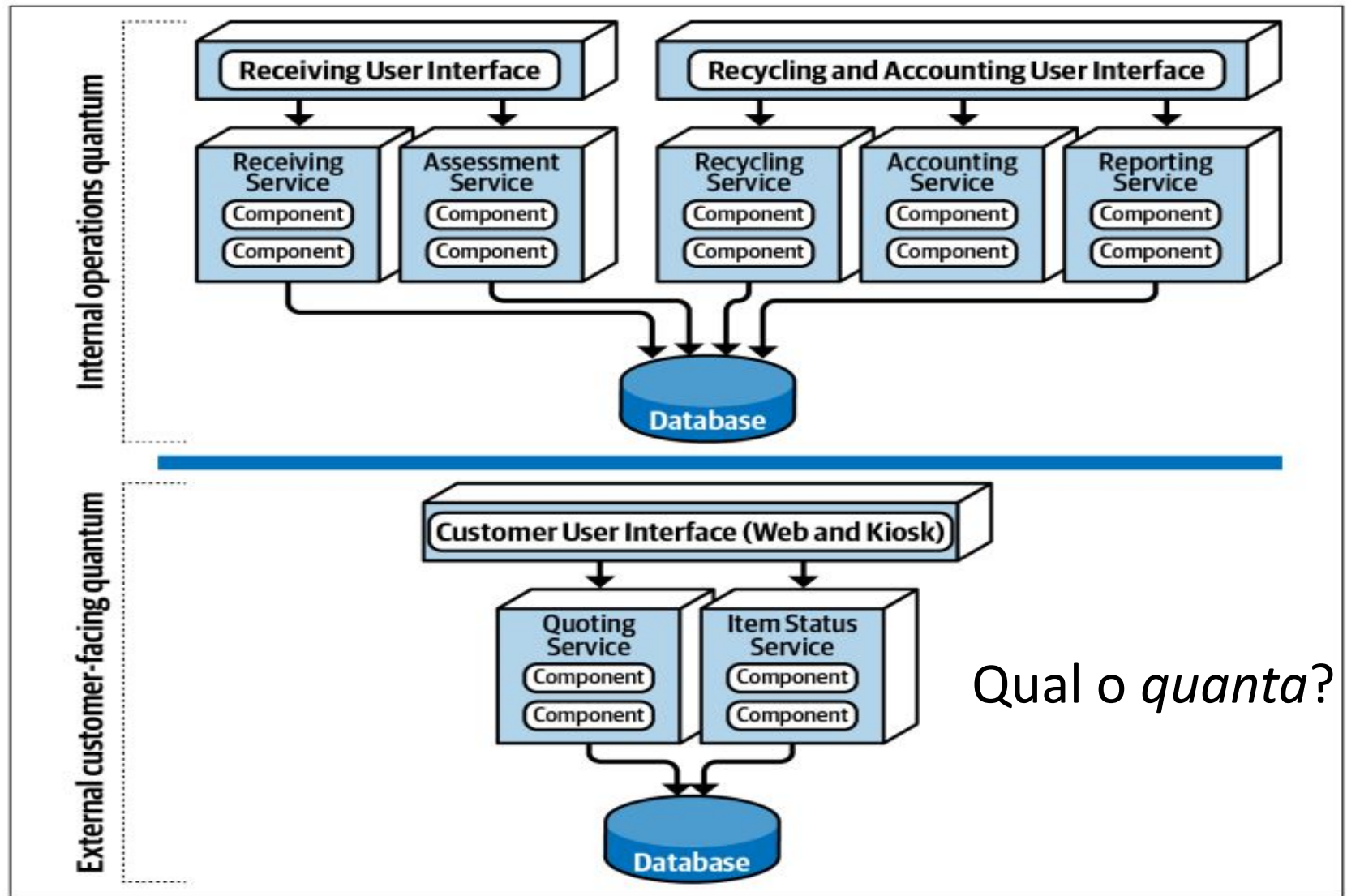


Figure 13-10. Separate quanta in a service-based architecture

Service-Based Architecture

- Onde podemos aplicar a arquitetura baseada em serviços?
- Cite alguns exemplos de aplicações que podem se beneficiar das características dessa arquitetura.

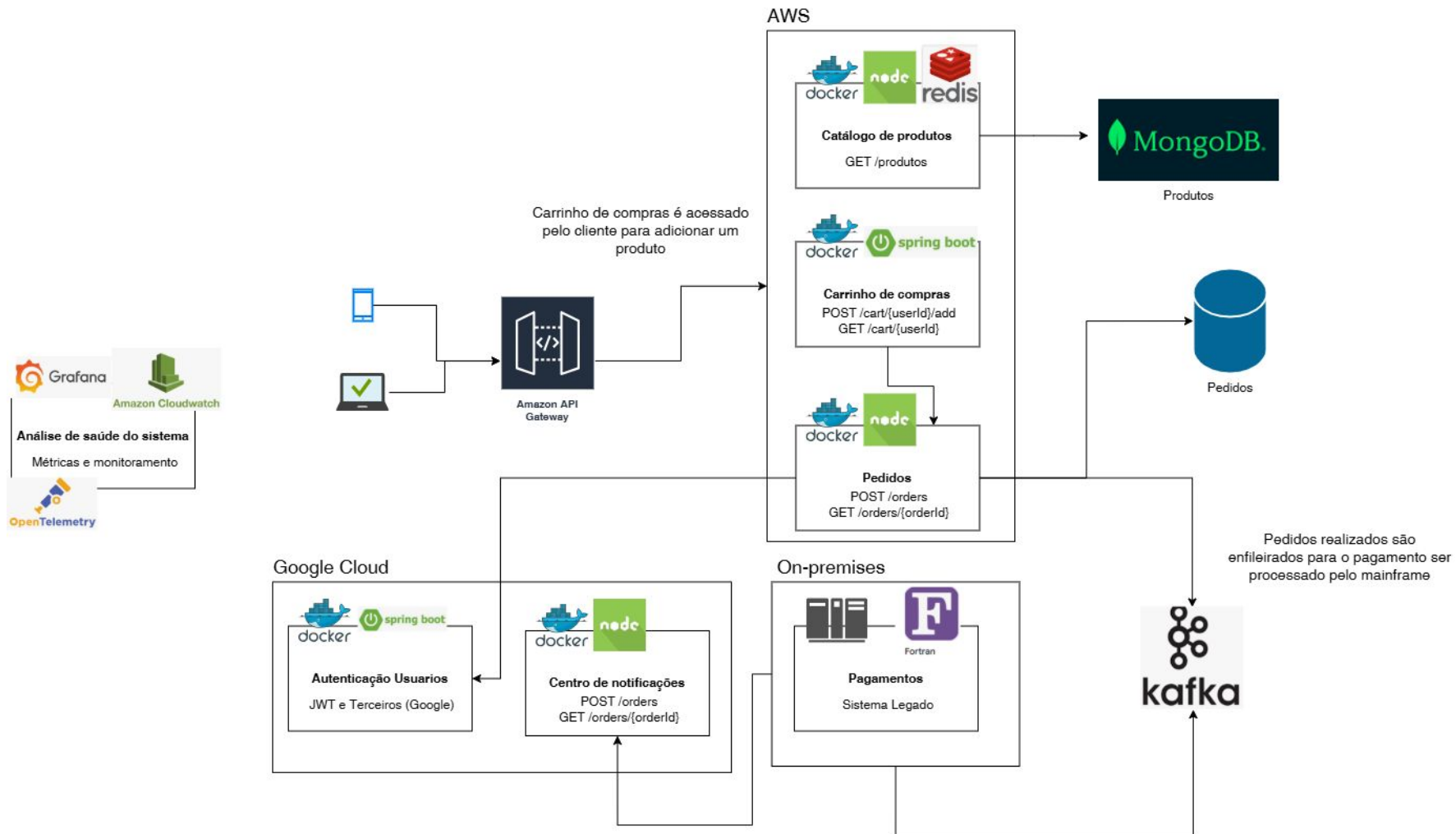
Service-Based Architecture

- A arquitetura baseada em serviços tem um bom balanceamento entre quase todas as características, sendo uma boa escolha para diversos nichos de aplicações.
- Porém, cuidado ao escolher essa arquitetura e o custo para criar e manter ser menor do que o benefício proporcionado.
- Cada domínio do sistema pode ser associado facilmente a um serviço, tornando essa arquitetura uma boa escolha para aplicações divididas por domínio.

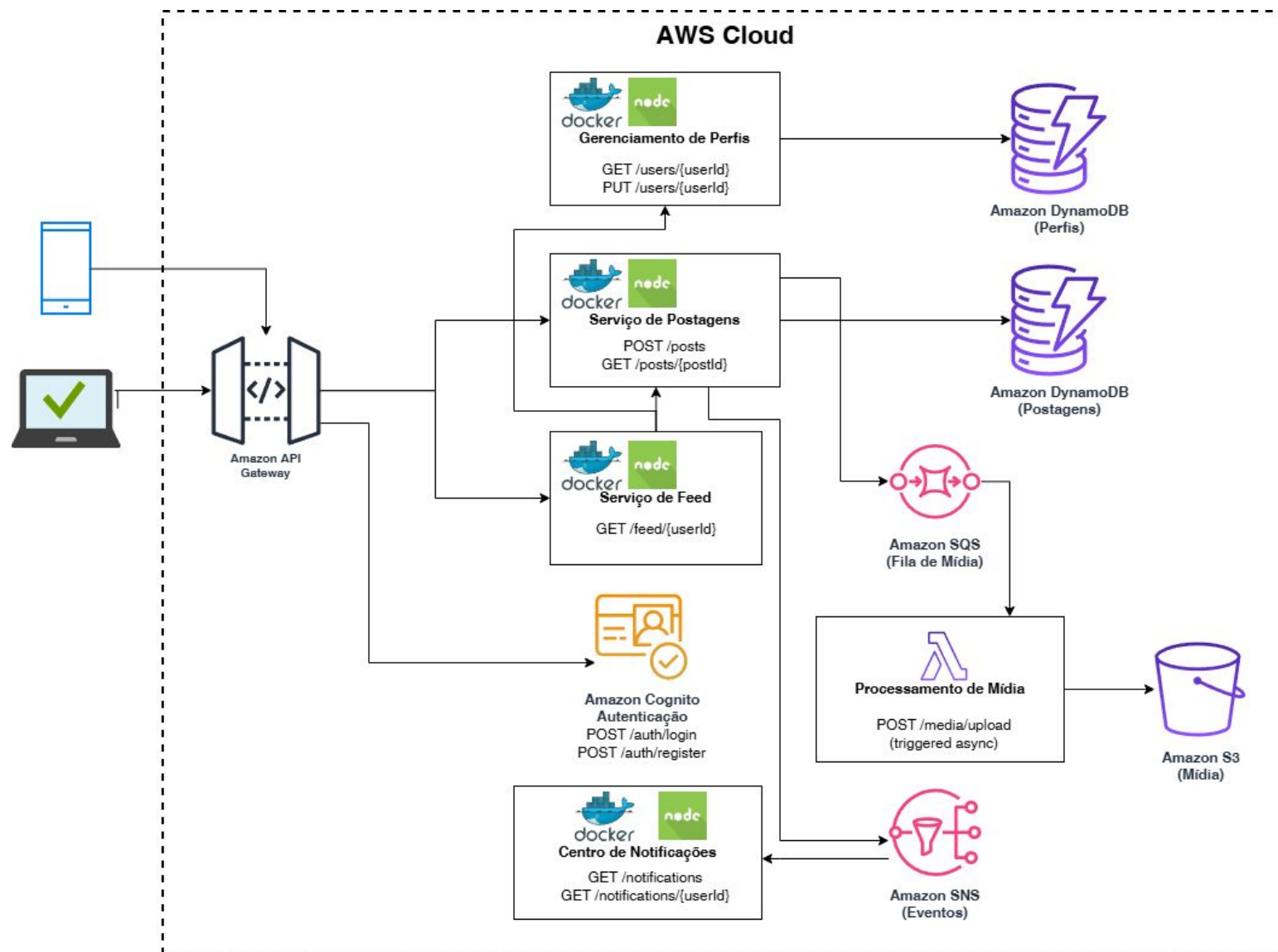
Service-Based Architecture

- Por fim, essa arquitetura não traz a granularidade fina dos microserviços (dezenas/centenas de serviços) e alcança uma modularidade adequada para a maioria das aplicações.
- Porém, se o seu domínio pode ser quebrado em muitos serviços e exige uma granularidade realmente fina, a arquitetura de microserviços pode ser uma alternativa mais adequada.

Service-Based Architecture



Service-Based Architecture



Dúvidas?



www.shutterstock.com · 744867163